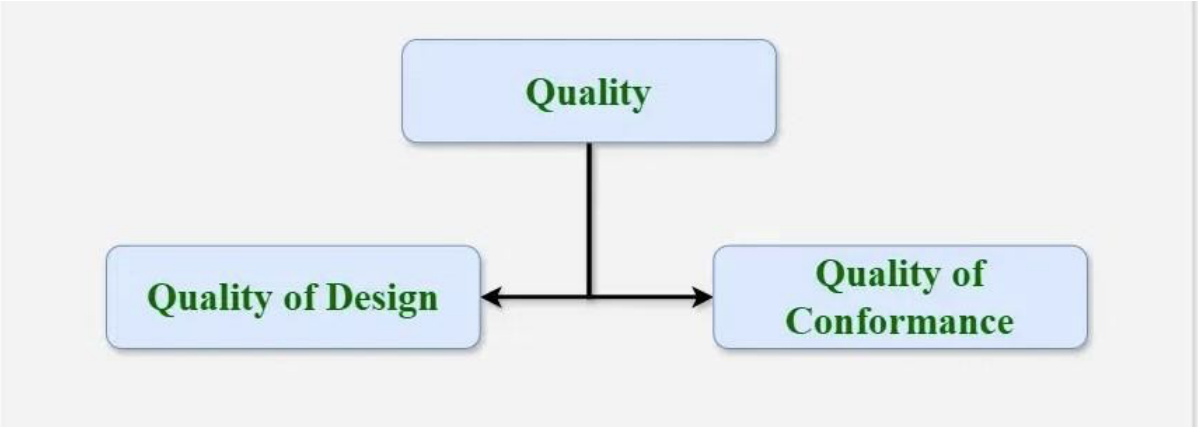
Software Quality Assurance

A Software Quality Assurance (SQA) is simply a way to assure quality in the software. It is the set of activities that ensure processes, procedures as well as standards are suitable for the project and implemented correctly. It is a process that works parallel to Software Development. It focuses on improving the process of development of software so that problems can be prevented before they become major issues. Software Quality Assurance is a kind of Umbrella activity that is applied throughout the software process.

Quality

Quality in a product or service can be defined by several measurable characteristics. Each of these characteristics plays a crucial role in determining the overall quality. Generally, the quality of the software is verified by third-party organizations like international standard organizations.



Quality in Software Engineering

Elements of Software Quality Assurance (SQA)

Software Quality Assurance (SQA) involves a comprehensive set of principles, practices, and activities aimed at ensuring that software products and processes consistently meet defined quality standards. It is beyond defect detection to encompass the proactive management, monitoring, and continuous improvement of software quality throughout the development lifecycle.

These can be summarized in the following manner

1. Standards
2. Reviews and Audits
3. Testing
4. Error/defect collection and analysis
5. Change Management
6. Education
7. Vendor Management
8. Security Management
9. Safety
10. Risk Management

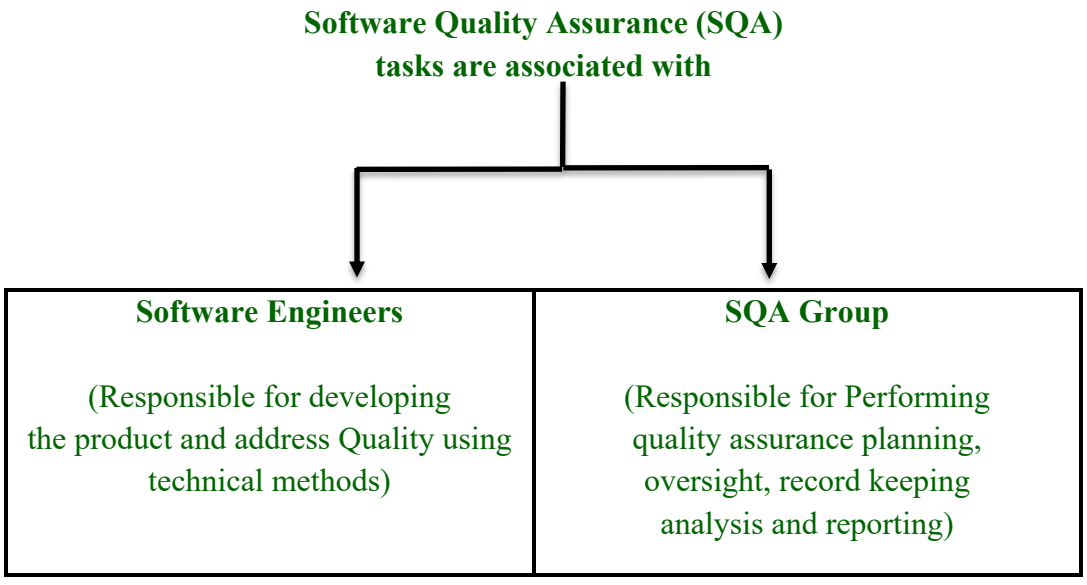
- **Standards:** The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.
- **Reviews and Audits:** Technical reviews are a quality control activity performed by software engineers for software engineers. Their intent is to uncover errors. Audits are a type of review performed by SQA personnel (people employed in an organization) with the intent of ensuring that quality guidelines are being followed for software engineering work.
- **Testing:** Software testing is a quality control function that has one primary goal to find errors. The job of SQA is to ensure that testing is properly planned and efficiently conducted for primary goal of software.
- **Error/defect collection and analysis:** SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.
- **Change Management:** Change is one of the most disruptive aspects of any software project. If it is not properly managed, change can lead to confusion, and confusion almost always leads to poor quality. SQA ensures that adequate change management practices have been instituted.
- **Education:** Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement which is key proponent and sponsor of educational programs.
- **Vendor Management:** Three categories of software are acquired from external software vendors shrink-wrapped packages, a tailored shell that provides a basic skeletal structure that is custom tailored to the needs of a purchaser, and contracted software that is custom designed and constructed from specifications provided by the customer organization. The job of the SQA organization is to ensure that high-quality software results by suggesting specific quality practices.

- **Security Management:** SQA ensures that appropriate process and technology are used to achieve software security.
- **Safety:** SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.
- **Risk Management:** The SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

SQA Tasks, Goals and Metrics

Software quality assurance is composed of a variety of tasks associated with two different constituencies—the software engineers who do technical work and an SQA group that has responsibility for quality assurance planning, oversight, record keeping, analysis, and reporting.

Software engineers address quality (and perform quality control activities) by applying solid technical methods and measures, conducting technical reviews, and performing well-planned software testing.



Software Tasks

SQA Tasks

The charter of the SQA group is to assist the software team in achieving a high-quality product. The Software Engineering Institute recommends a set of SQA actions that address quality assurance planning, oversight, record keeping, analysis, and reporting.

These actions are performed by an independent SQA group that:

- **Prepares an SQA plan for a project.** The plan is developed as part of project planning and is reviewed by all stakeholders. Quality assurance actions performed by the software engineering team and the SQA group are governed by the plan. The plan identifies evaluations to be performed, audits and reviews to be conducted, standards that are applicable to the project, procedures for error reporting and tracking, work products that are produced by the SQA group, and feedback that will be provided to the software team.
- **Participates in the development of the project's software process description.** The software development team chooses an appropriate process to execute the planned work. The SQA group then examines this process description to ensure it aligns with the organization's policies, adheres to internal software standards, meets externally mandated standards such as ISO-9001, and complies with other relevant elements of the overall software project plan.
- **Reviews software engineering activities to verify compliance with the defined software process.** The SQA group detects, documents, and monitors any deviations from the defined process, and ensures that the necessary corrective actions are implemented and verified.
- **Audits designated software work products to verify compliance with those defined as part of the software process**. The SQA group reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made; and periodically reports the results of its work to the project manager.
- Ensures that deviations in software work and work products are documented and handled according to a documented procedure.
- Records any noncompliance and reports to senior management. Noncompliance items are tracked until they are resolved.

Goals, Attributes and Metrics

The Software Quality Assurance (SQA) actions described earlier are carried out with the goal of ensuring software products meet high-quality standards throughout the development lifecycle. These actions align with a set of pragmatic quality goals, which focus on key areas that influence the overall quality of the software.

- **Requirements quality**: The correctness, completeness, and consistency of the requirements model will have a strong influence on the quality of all work products that follow. SQA must ensure that the software team has properly reviewed the requirements model to achieve a high level of quality.

  Attributes of Requirement Quality

  1. **Ambiguity** – Measures how unclear the requirements are. Ambiguous terms like many, large, or user-friendly can be interpreted differently, leading to misunderstandings.
     Metric: Count of vague modifiers.
  2. **Completeness** – Checks whether all required details are provided in the requirements. Missing parts (e.g., TBA – To Be Added, TBD – To Be Determined) indicate incomplete requirements.
     Metric: Number of TBA/TBD instances.
  3. **Understandability** – Assesses how easy it is for stakeholders to read and comprehend requirements.
     Metric: Number of sections/subsections (well-structured documents are easier to follow).
  4. **Volatility** – Tracks how often requirements change during the project, which can impact stability.
     Metric: Number of changes per requirement and time taken for each change request.
  5. **Traceability** – Measures the ability to link each requirement to design/code components. Poor traceability makes verification and maintenance difficult.
     Metric: Number of requirements not linked to design/code.
  6. **Model Clarity:** The design should be easy to understand and well-documented using diagrams and descriptions.
     Metric: Number of UML models, descriptive pages per model.

- **Design quality**: Every element of the design model should be assessed by the software team to ensure that it exhibits high quality and that the design itself conforms to requirements. SQA looks for attributes of the design that are indicators of quality.

  Attributes of Design Quality

  1. **Architectural Integrity:** Ensures the design follows a coherent architecture and defined principles.
     Metric: Presence of an architectural model
  2. **Component Completeness:** Checks whether all components are properly defined and connected in the design.
     Metric: Number of components linked to the architecture model, Complexity.

3. **Interface Complexity:** Evaluates how complicated the interaction between components is. High complexity can reduce maintainability.
   Metrics: Average number of picks to get to a typical function or content , Layout appropriateness
4. **Patterns:** Measures the use of proven design patterns to improve maintainability, reusability and structure.
   Metrics: Number of patterns used.

- **Code quality**: Source code and related work products must conform to local coding standards and exhibit characteristics that will facilitate maintainability. SQA should isolate those attributes that allow a reasonable analysis of the quality of code.

Attributes of Code Quality

1. **Complexity:** How difficult the code is to read and maintain.
   Metrics: Cyclomatic complexity (Number of independent paths in the code).
2. **Maintainability:** How easy it is to modify and fix the code without introducing new errors.
   Metrics: Design Factors.
3. **Understandability:** How easily a developer can read and grasp the code's logic.
   Metrics: Percentage of comments, Variable naming conventions
4. **Reusability:** The extent to which existing components are reused.
   Metrics: Percentage of reused components.
5. **Documentation:** Quality and readability of technical documentation.
   Metrics: Readability Index.

- **Quality control effectiveness**. A software team should apply limited resources in a way that has the highest likelihood of achieving a high-quality result. SQA analyzes the allocation of resources for reviews and testing to assess whether they are being allocated in the most effective manner.

Attributes of Quality Control Effectiveness

1. **Resource Allocation:** Whether the right amount of effort and time is allocated to quality tasks.
   Metrics: Staff hour percentage per activity.
2. **Completion Rate:** Measures adherence to planned timelines.
   Metrics: Actual vs budgeted completion time.
3. **Review Effectiveness:** Determines how well reviews (Code, Design, Requirements) identify issues.
   Metrics: See review metrics.
4. **Testing effectiveness:** Evaluates how well testing finds and addresses errors.
   Metrics: Number of errors found, severity, and effort required to fix them.

Goal	Attribute	Metric
Requirement Quality	Ambiguity	Number of ambiguous modifiers (e.g., many, large, human-friendly)
	Completeness	Number of TBA, TBD
	Understandability	Number of sections/subsections
	Volatility	Number of changes per requirement.
		Time (by activity) when change is requested
	Traceability	Number of requirements not traceable to design/code
	Model clarity	Number of UML models
		Number of descriptive pages per model
		Number of UML errors
Design Quality	Architectural integrity	Existence of architectural model
	Component completeness	Number of components that trace to architectural model
		Complexity of procedural design
	Interface complexity	Average number of picks to get to a typical function
		Layout appropriateness
	Patterns	Number of patterns used
Code quality	Complexity	Cyclomatic complexity
	Maintainability	Design factors
	Understandability	Percent internal comments
		Variable naming conventions
	Reusability	Percent reused components
	Documentation	Readability index
QC effectiveness	Resource	allocation Staff hour percentage per activity
	Completion rate	Actual vs. budgeted completion tim
	Review effectiveness	See review metrics
	Testing effectiveness	Number of errors found and criticality
		Effort required to correct an error
		Origin of error

Formal Approaches to SQA

Software quality is everyone's job, and it can be achieved through competent software engineering practice as well as through the application of technical reviews, a multi-tiered testing strategy, better control of software work products and the changes made to them, and the application of accepted software engineering standards. In addition, quality can be defined in terms of a broad array of quality attributes and measured using a variety of indices and metrics.

Software engineering community has argued that a more formal approach to software quality assurance is required.

Some of the arguments of SQA:

1. Computer program is mathematical object.
2. A rigorous syntax and semantics can be defined for every programming language and a rigorous approach to the specification of software requirements Is available.
3. If the requirements model and the programming language can be represented in a rigorous manner it should be possible to apply mathematical proof of correctness to demonstrate that a program conforms exactly to its specifications.

Key Formal Approaches in SQA

Standards and Guidelines Compliance: Following established standards like ISO/IEC 25010 or CMMI provides a framework for quality and consistency in software development.

Quality, Audits and Reviews

- Technical Reviews: Evaluating design documents and other technical artifacts.
- Inspections: Formal examination of software products to identify defects.
- Walkthroughs: Peer group discussions to review work products.

Software Testing

- **Unit Testing:** Testing individual components.
- **Integration Testing:** Testing how different modules work together.
- **System Testing:** Testing the entire system.
- **Acceptance Testing:** Validating software against user requirements.

Six Sigma for Software Engineering

Six Sigma is currently one of the most widely adopted strategies for statistical quality assurance in the industry. First introduced and popularized by Motorola in the 1980s, it is defined as a disciplined, data-driven methodology that applies statistical analysis to measure and enhance a company's operational performance by detecting and eliminating defects in both manufacturing and service-oriented processes.

The name Six Sigma refers to a process capability level of six standard deviations, which equates to only 3.4 defects per million opportunities, representing an exceptionally high-quality benchmark.

The Six Sigma approach is built around three core steps:

- Define customer requirements and deliverables and project goals via well-defined methods of customer communication.
- Measure the existing process and its output to determine current quality performance.
- Analyze defect metrics and determine the vital few causes.

If a software process already exists but requires enhancement, Six Sigma recommends two additional steps:

- Improve the process by eliminating the root causes of defects.
- Control the process to ensure that future work does not reintroduce the causes of defects.

These core steps of six sigma is sometimes referred to as **DMAIC**

D -> Define: Clearly define the problem, Project Goals, and customer requirements.

M -> Measure: Collect data to understand the current process performance and identify areas of variation.

A-> Analyze: Analyze the data to determine the root causes of the problems and defects.

I -> Improve: Implement solutions to eliminate the root causes and improve the process.

C -> Control: Establish controls to sustain the improvements and prevent future defects.

If an organization is developing a software process (rather than improving an existing process), the core steps are augmented as follows:

- Design the process to avoid the root causes of defects and to meet customer requirements.
- Verify that the process model will, in fact, avoid defects and meet customer requirements.

This variation is sometimes called the DMADV (define, measure, analyze, design, and verify) method.

Define: Define the project goals and customer requirements.

Measure: Measure customer needs and specifications.

Analyze: Analyze the data to identify potential solutions.

Design: Design a new process or product to meet customer needs.

Verify: Verify the design to ensure it meets the requirements.

Benefits of Six Sigma

1. **Reduced Defects**: Minimizes errors and defects in processes.
2. **Improved Quality**: Enhances the quality of products and services.
3. **Increased Efficiency**: Streamlines processes and reduces waste.
4. **Reduced Costs**: Saves money by eliminating defects and improving efficiency.
5. **Improved Customer Satisfaction**: Meeting customer requirements leads to increased satisfaction.
6. **Enhanced Employee Morale:** Involving employees in the improvement process can boost morale.

Software Reliability

One of the most important elements of computer program is reliability. If a program repeatedly and frequently fails to perform, it matters little whether other software quality factors are acceptable.

Software reliability, unlike many other quality factors, can be measured directly and estimated using historical and developmental data. Software reliability is defined in statistical terms as **"the probability of failure-free operation of a computer program in a specified environment for a specified time".**

In the context of software quality and reliability, a failure is defined as any deviation from specified software requirements. However, even within this definition, failures can vary significantly in nature and impact.

Some failures may be merely inconvenient, while others can be catastrophic. Certain issues can be resolved within seconds, whereas others might take weeks or even months to address. Adding to the complexity, fixing one failure can sometimes introduce new errors, which may lead to additional failures down the line.

In essence, understanding software failures requires not only recognizing deviations from requirements but also considering their severity, resolution time, and potential ripple effects on the system.

Measures of Reliability and Availability

Early research in software reliability sought to adapt the mathematical models of hardware reliability theory for predicting software reliability. However, most hardware reliability models are based on failures caused by physical wear and tear, rather than design defects. In hardware, issues such as temperature effects, corrosion, and mechanical shock are far more common causes of failure than design flaws.

In contrast, software behaves differently—every software failure stems from design or implementation errors, as software does not degrade physically over time.

This distinction has fueled an ongoing debate about how well key principles of hardware reliability apply to software. While no definitive connection has been proven, it is still valuable to examine certain fundamental concepts that are relevant to both hardware and software systems.

If we consider a computer-based system, a simple measure of reliability is meantime-between- failure (MTBF):

MTBF = MTTF+MTTR

Where MTTF and MTTR are mean-time-to-failure and mean-time-to-repair.

Many researchers consider **Mean Time Between Failures (MTBF)** to be a more practical and meaningful indicator of software reliability than other quality-related metrics. From the end user's perspective, **the primary concern is the occurrence of failures, not the total number of defects** in the software.

This is because not all defects lead to failures at the same rate. The total defect count, therefore, offers little insight into the actual reliability of the system.

Despite its usefulness, MTBF has two limitations:

1. It estimates the time interval between failures but does not directly provide the projected failure rate.
2. It is often misinterpreted as representing the average lifespan of the software, which is not its actual meaning.

An alternative reliability measure is Failures-In-Time (FIT), which represents the expected number of failures in one billion hours of operation. 1 FIT equal one failure per billion operational hours.

As we have reliability measure, we also should develop a measure of availability. The system availability is the probability that a program is operating according to requirements at a given point in time and is defined as

$$\text{Availability} = \frac{MTTF}{MTTF+MTTR} \text{ X } 100\%$$

Software Safety

Software safety is a key aspect of software quality assurance that concentrates on identifying and evaluating potential hazards that could adversely impact the software and possibly lead to complete system failure. Detecting such hazards early in the software development process allows designers to incorporate specific features into the software that can either eliminate these risks entirely or control them effectively to minimize their impact.

As part of software safety, a structured modelling and analysis process is carried out. In the initial stage, potential hazards are systematically identified and then classified based on their criticality (the severity of their potential impact) and risk (the likelihood of their occurrence).

Example – Cruise Control System Hazards

To understand hazard identification in software safety, consider a computer-based cruise control system in an automobile. Possible hazards might include:

1. Uncontrolled acceleration that cannot be stopped – This is a critical safety risk because it could lead to accidents and injuries if the driver is unable to regain control.
2. Failure to disengage when the brake pedal is pressed – The system should automatically turn off when braking, so ignoring this input can cause dangerous situations.
3. Failure to engage when the activation switch is pressed – While not life-threatening, this affects functionality and driver convenience.
4. Gradual loss or gain of speed – A subtle hazard that may not cause immediate danger but can reduce system reliability and driver trust.

Once such system-level hazards are identified, analysis techniques—such as Failure Mode and Effects Analysis (FMEA) or Fault Tree Analysis (FTA)—are applied to evaluate:

- Severity – How serious the impact would be if the hazard occurred.
- Probability of occurrence – The likelihood of the hazard happening during system operation.

Analysis Techniques

Common methods to evaluate these hazard chains and their likelihood include:

- **Fault Tree Analysis (FTA)** – Identifies potential failure paths leading to hazards.
- **Real-Time Logic** – Models timing-related hazards and dependencies.
- **Petri Net Models** – Represents event sequences and concurrency in hazard formation.

Software Reliability and Software Safety

- Software Reliability – Uses statistical methods to estimate the likelihood of software failures.
- Software Safety – Focuses on understanding how failures could lead to hazardous conditions or mishaps. Failures are evaluated in the full context of the system and its environment, rather than in isolation.

Few Examples of Safety-Critical Software Systems:

- **Automotive:** Anti-lock braking systems (ABS), airbag control systems, electronic stability control (ESC), and adaptive cruise control.
- **Aerospace:** Flight control systems, navigation systems, and engine control systems in aircraft.
- **Medical Devices:** Pacemakers, insulin pumps, and surgical robots.

Software Safety Principles and Practices

- Hazard Analysis: Identifying potential hazards and risks associated with the software system.
- Secure Coding Practices: Following guidelines and standards to minimize vulnerabilities and errors in the code.
- Error Handling and Recovery: Implementing mechanisms to gracefully handle errors and prevent system failures.
- Testing and Verification: Thoroughly testing the software to ensure it meets safety requirements and behaves as expected under various conditions.
- Redundancy and Fail-safe Mechanisms: Incorporating redundant systems and fail- safe mechanisms to provide backup in case of failure.
- Access Control: Implementing strong access controls to protect sensitive data and functionality.
- Regular Security Audits and Updates: Regularly reviewing and updating the software to address vulnerabilities and maintain security.
- Security Training: Providing security awareness training to developers and users to promote safe practices.
- Incident Response Planning: Having a plan in place to respond to security incidents and minimize their impact.

ISO 9000 Quality Standards

ISO (International Standards Organization) is a group or consortium of 63 countries established to plan and fosters standardization. ISO declared its 9000 series of standards in 1987. It serves as a reference for the contract between independent parties. The ISO 9000 standard determines the guidelines for maintaining a quality system. The ISO standard mainly addresses operational methods and organizational methods such as responsibilities, reporting, etc. ISO 9000 defines a set of guidelines for the production process and is not directly concerned about the product itself.

Types of ISO 9000 Quality Standards

1. **The ISO 9000** series of standards assumes that if a proper stage is followed for production, then good quality products are bound to follow automatically. The types of industries to which the various ISO standards apply are as follows.
2. **ISO 9001**: This standard applies to the organizations engaged in design development, production, and servicing of goods. This is the standard that applies to most software development organizations.
3. **ISO 9002**: This standard applies to those organizations which do not design products but are only involved in the production. Examples of these category industries contain steel and car manufacturing industries that buy the product and plants designs from external sources and are engaged in only manufacturing those products. Therefore, ISO 9002 does not apply to software development organizations.
4. **ISO 9003**: This standard applies to organizations that are involved only in the installation and testing of the products. For example, Gas companies.

An organization determines to obtain ISO 9000 certification applies to ISO registrar office for registration.

Process to Obtain ISO Certificate

1. **Application:** Once an organization decided to go for ISO certification, it applies to the registrar for registration.
2. **Pre-Assessment:** During this stage, the registrar makes a rough assessment of the organization.
3. **Document review and Adequacy of Audit:** During this stage, the registrar reviews the document submitted by the organization and suggest an improvement.
4. **Compliance Audit:** During this stage, the registrar checks whether the organization has compiled the suggestion made by it during the review or not.
5. **Registration:** The Registrar awards the ISO certification after the successful completion of all the phases.
6. **Continued Inspection:** The registrar continued to monitor the organization time by time.